

EXPLICACIÓ SIOP CHALLENGE - DAVID PÉREZ CRUZ

CODI TT.C:

En aquesta part del codi es defineixen diverses constants, com ara `SERVNAME`, `SERVADDR`, `SERVPORT`, i `CTRLPORT`, que configuren la connexió amb el servidor. A més, es defineixen credencials, com `TEAMNAME`, `ENIGMA3`, `SILVERKEY` i `GOLDKEY`, que seran utilitzades per autenticar la comunicació amb el servidor. També s'estableix un límit màxim de buffer a través de la constant `MAX_BUFFER`, que ajuda a prevenir desbordaments de memòria en la manipulació de dades.

La variable global `replicants_preparats`, inicialitzada a 0, serveix per comptar quants processos replicants han notificat estar llestos mitjançant senyals. Aquesta variable es declara com a `volatile sig_atomic_t` per garantir un accés segur dins del gestor de senyals i evitar comportaments indefinits causats per l'optimització del compilador.

El gestor de senyal, `gestor_senyal`, està dissenyat per gestionar la senyal `SIGUSR1`. Quan aquesta senyal es rep, incrementa la variable `replicants_preparats`, indicant que un altre replicant ha notificat estar preparat. Aquest mecanisme és essencial per la sincronització i coordinació entre els processos replicants i el procés principal.

La funció `valida_enter_positiu` valida que una cadena donada com a entrada representa un número enter positiu. Això és útil per assegurar que els paràmetres passats al programa són correctes i compleixen els requisits establerts.

Per altra banda, la funció `gestiona_error` proporciona un mecanisme robust per gestionar errors crítics. Quan es detecta un error, mostra un missatge descriptiu mitjançant la funció `perror` i finalitza el programa amb un codi de sortida que indica fallada. Aquest enfocament garanteix que el programa no continuï en cas d'una situació invàlida, millorant la fiabilitat i la seguretat del codi.

```
/* Programa robust: tt_.c */  
#include "dummy4.h"  
#include <stdio.h>
```

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>

#define SERVNAME "SIOP-server"
#define SERVADDR "147.83.159.200"
#define SERVPORT 9999
#define CTRLPORT 9900
#define TEAMNAME "davidperezcruz"
#define ENIGMA3 "belters"
#define SILVERKEY "ab452"
#define GOLDKEY ""
#define MAX_BUFFER 200

volatile sig_atomic_t replicants_ready = 0; // Variable per
comptar els replicants preparats

// Manejador de la senyal SIGUSR1
void signal_handler(int sig) {
    if (sig == SIGUSR1) {
        replicants_ready++; // Incrementa el comptador quan rep
una senyal
    }
}

// Funció per validar que un valor és un enter positiu
int validate_positive_int(char *str) {
    int num = atoi(str); // Converteix la cadena a enter
    return num > 0; // Retorna veritable si el número és positiu
}

// Funció per gestionar errors i mostrar un missatge d'error
void handle_error(const char *message) {
    perror(message); // Mostra l'error
    exit(EXIT_FAILURE); // Termina el programa amb codi d'error
}
```

n aquesta part del codi, la funció main comença validant els paràmetres que es passen a través de la línia de comandes. Si no es passen exactament dos arguments (a part del nom del programa), es mostra un missatge d'ús correcte i el programa finalitza amb un codi d'error. Aquest control assegura que els paràmetres siguin els esperats per al correcte funcionament del programa.

A continuació, el codi comprova que els dos paràmetres passats (M i N) siguin enters positius. La funció `valida_enter_positiu` s'utilitza per verificar que ambdós valors representin números majors que 0. Si qualsevol d'ells no compleix aquesta condició, es mostra un missatge d'error i el programa es deté. Aquesta validació és crucial per evitar situacions de comportament indefinit o errors més endavant.

Després de la validació, els valors de M i N es converteixen de cadenes a enters utilitzant la funció `atoi`. El programa també comprova que el valor de N no sigui superior al límit màxim definit per `MAX_BUFFER`. Si aquest límit es supera, el programa mostra un missatge d'error i finalitza per evitar desbordaments de memòria.

Una vegada que els paràmetres són vàlids, el programa estableix connexions amb dos servidors. Primer, es realitza la connexió al servidor de control mitjançant la funció `ctrlconnect`. Si la connexió falla (el retorn és menor que 0), s'executa la funció `gestiona_error`, que mostra el missatge d'error i finalitza el programa. El mateix procés es repeteix per a la connexió al servidor de dades mitjançant la funció `connecta`.

Després de les connexions, es configura un gestor per a la senyal `SIGUSR1` utilitzant la funció `signal`. Quan es rep aquesta senyal, el programa incrementarà el comptador de replicants preparats, una funcionalitat clau per a la sincronització entre processos.

A continuació, el programa crea M replicants mitjançant un bucle `for`. Per a cada iteració, es crida a `fork()` per crear un nou procés fill. Si la crida a `fork` retorna 0 (indicant que estem dins del procés fill), es redirigeix l'entrada estàndard perquè apunti al descriptor de fitxer de dades (`fd_dades`), que establirà la connexió amb el servidor de dades. Això es fa tancant l'entrada estàndard (`close(0)`) i duplicant el

descriptor de fitxer de dades (dup(fd_dades)) abans de tancar el descriptor original (close(fd_dades)).

Després, el programa converteix el valor de N a cadena mitjançant la funció snprintf, i executa el programa ./replicant amb aquest valor com a paràmetre a través de la funció execlp. Si hi ha algun error en executar el replicant, es crida a gestiona_error per gestionar l'error de manera robusta.

Si la crida a fork retorna un valor negatiu (indicant un error en la creació del procés), també es crida a la funció gestiona_error per acabar el programa de manera segura.

```
int main(int argc, char *argv[]) {
    if (argc != 3) { // Comprova que es passen els paràmetres
correctes
        fprintf(stderr, "Ús: ./tt M > 0 N > 0\n");
        exit(EXIT_FAILURE);
    }

    // Comprova que M i N siguin enters positius
    if (!valida_enter_positiu(argv[1]) ||
!valida_enter_positiu(argv[2])) {
        fprintf(stderr, "Error: M i N han de ser enters majors que
0.\n");
        exit(EXIT_FAILURE);
    }

    // Assigna els valors de M i N
    int M = atoi(argv[1]);
    int N = atoi(argv[2]);
    if (N > MAX_BUFFER) { // Comprova que N no superi el límit
màxim
        fprintf(stderr, "Error: N no pot excedir %d.\n",
MAX_BUFFER);
        exit(EXIT_FAILURE);
    }

    // Connexió al servidor de control
    int fd_control = ctrlconnect(SERVADDR, CTRLPORT, ENIGMA3,
TEAMNAME, SILVERKEY, GOLDKEY);
    if (fd_control < 0) {
```

```

        gestiona_error("Error connectant al Capitol (canal de
control)");
    }

    // Connexió al servidor de dades
    int fd_dades = connecta(SERVADDR, SERVPORT);
    if (fd_dades < 0) {
        gestiona_error("Error connectant al Capitol (canal de
dades)");
    }

    signal(SIGUSR1, gestor_senyal); // Assigna el gestor de la
senyal

    // Creació de M replicants
    for (int i = 0; i < M; i++) {
        int pid = fork(); // Crea un nou procés
        if (pid == 0) { // Codi del replicant
            close(0); // Tanca l'entrada estàndard
            dup(fd_dades); // Duplica el descriptor de fitxer per
connectar-se
            close(fd_dades); // Tanca el descriptor original

            char N_cadena[16];
            snprintf(N_cadena, sizeof(N_cadena), "%d", N); //
Converteix N a cadena

            execlp("./replicant", "./replicant", N_cadena, NULL);
// Executa el replicant
            gestiona_error("Error executant replicant");
        } else if (pid < 0) { // Error en la creació del procés
            gestiona_error("Error creant replicant");
        }
    }
}

```

Aquesta part del codi es centra en l'espera dels processos replicants, l'acumulació dels seus resultats i la comunicació amb el servidor de control.

Primer, es defineix una variable acumulador inicialitzada a 0. Aquesta variable s'utilitza per acumular els resultats retornats pels processos replicants, que es calculen al llarg de l'execució del programa.

A continuació, es fa un bucle que s'executa M vegades, una per cada procés replicant creat anteriorment. La funció `wait(&estat)` és cridada per esperar que cada procés fill (un replicant) acabi la seva execució. Quan un procés fill finalitza, `wait` retorna un valor que indica l'estat de sortida del procés, el qual es guarda a la variable `estat`.

Es verifica si el procés fill ha finalitzat correctament mitjançant la macro `WIFEXITED(estat)`. Si aquesta condició es compleix, es recupera el codi de sortida del procés mitjançant `WEXITSTATUS(estat)` i s'afegeix a la variable acumulador. Aquest valor representa el resultat del replicant, i la seva acumulació permet obtenir el resultat global de la tasca. Si el procés fill no ha acabat correctament (és a dir, no ha sortit amb un codi d'èxit), es mostra un missatge d'error indicant quina replicant ha fallat, i el programa finalitza amb un codi d'error utilitzant `exit(EXIT_FAILURE)`.

Un cop s'han esperat tots els replicants i s'ha acumulat el resultat, el programa prepara la comunicació amb el servidor de control. Es crea un buffer `buf` de 128 bytes per emmagatzemar el resultat acumulat com a cadena. Utilitzant `snprintf`, es converteix el valor de acumulador en una cadena de caràcters i es guarda a `buf`. A continuació, la funció `write(fd_control, buf, nbytes)` s'encarrega d'enviar aquest resultat al servidor de control, on `fd_control` és el descriptor de fitxer associat a la connexió amb el servidor. Si la crida a `write` retorna un valor negatiu, significa que hi ha hagut un error en l'escriptura, i es crida la funció `gestiona_error` per gestionar l'error de manera robusta.

Després de enviar el resultat al servidor de control, el programa llegeix la resposta del servidor amb la funció `read(fd_control, resposta, sizeof(resposta) - 1)`. El contingut de la resposta es desa a la variable `resposta`, i el valor llegit es guarda a la variable `len`. Si la crida a `read` retorna un valor negatiu, significa que ha ocorregut un error en llegir la

resposta, i el programa finalitza amb un error a través de la funció `gestiona_error`.

Finalment, el programa afegeix un caràcter nuls (`\0`) al final de la cadena llegida per assegurar que es tracta correctament com a cadena de caràcters i imprimeix la resposta del servidor mitjançant `printf("MISSION: %s\n", resposta)`. Això mostra el missatge rebut del servidor, que pot contenir la informació sobre l'estat de la missió o altres dades rellevants.

Per acabar, es tanquen les connexions amb els servidors de control i de dades mitjançant les funcions `close(fd_control)` i `close(fd_dades)`, alliberant els recursos associats a aquestes connexions. El programa finalitza retornant 0, indicant que s'ha executat amb èxit.

```
int acumulador = 0;
// Espera els resultats de tots els replicants
for (int i = 0; i < M; i++) {
    int estat;
    wait(&estat); // Espera que el procés fill acabi

    if (WIFEXITED(estat)) {
        acumulador += WEXITSTATUS(estat); // Acumula els
resultats
    } else {
        fprintf(stderr, "Replicant %d no va acabar
correctament.\n", i);
        exit(EXIT_FAILURE);
    }
}

// Envia el resultat acumulat al servidor de control
char buf[128];
int nbytes = snprintf(buf, sizeof(buf), "%d", acumulador);
if (write(fd_control, buf, nbytes) < 0) {
    gestiona_error("Error escrivint al Capitol (control)");
}

// Llegeix la resposta del servidor de control
char resposta[128];
int len = read(fd_control, resposta, sizeof(resposta) - 1);
if (len < 0) {
```

```
        gestiona_error("Error llegint del Capitol (control)");
    }
    resposta[len] = '\0';
    printf("MISSION: %s\n", resposta); // Imprimeix la resposta

    close(fd_control); // Tanca la connexió al servidor de control
    close(fd_dades); // Tanca la connexió al servidor de dades
    return 0;
}
```

CODI REPLICANT.C:

Aquesta part del codi es centra en la gestió de les entrades proporcionades per l'usuari, la validació dels paràmetres i la inicialització de la connexió al servidor.

En primer lloc, es comprova que el nombre d'arguments passats al programa sigui el correcte. El programa espera rebre exactament un paràmetre addicional, N, que indica la mida del buffer que el replicant utilitzarà. Si no es passa el nombre correcte de paràmetres, el programa mostra un missatge d'error que explica com ha de ser l'ús correcte de la comanda i finalitza l'execució amb un codi d'error utilitzant `exit(EXIT_FAILURE)`.

Després, el programa valida que el paràmetre N sigui un enter positiu mitjançant la funció `valida_enter_positiu`. Aquesta funció utilitza la funció `atoi` per convertir la cadena de caràcters representant el valor de N a un enter i comprova que sigui superior a 0. Si N no és un enter positiu, el programa mostra un missatge d'error i finalitza l'execució amb un codi d'error.

A continuació, es comprova que el valor de N no superi el límit màxim de mida de buffer definit per la constant `MAX_BUFFER`. Si N és més gran que aquest límit, es mostra un missatge d'error indicant que N no pot excedir aquest valor màxim, i el programa també finalitza amb un codi d'error.

Per últim, es crida a la funció `dummy_init` (probablement definida en la biblioteca "dummy4.h") per inicialitzar la connexió amb el servidor utilitzant el nom de servidor `SERVNAME` i el límit màxim de buffer `MAX_BUFFER`. Aquest pas prepara el programa per interactuar amb el servidor de dades que serà utilitzat durant l'execució.

En resum, aquesta part del codi gestiona les entrades proporcionades per l'usuari, validant-les i comprovant-les per garantir que siguin correctes abans de passar a la següent fase d'inicialització de la connexió amb el servidor. Si alguna de les condicions no es compleix, el programa mostra un missatge d'error i finalitza amb un codi d'error per evitar un comportament incorrecte durant l'execució.

```
/* Programa robust per replicant.c */
```

```

#include "dummy4.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>

#define MAX_BUFFER 200
#define SERVNAME "SIOP-server"
#define TEAMNAME "davidperezcruz"
#define ENIGMA3 "belters"

// Funció per gestionar errors
void gestiona_error(const char *missatge) {
    perror(missatge); // Mostra l'error
    exit(EXIT_FAILURE); // Termina el programa amb codi d'error
}

// Funció per validar que un valor és un enter positiu
int valida_enter_positiu(char *cadena) {
    int num = atoi(cadena); // Converteix la cadena a enter
    return num > 0; // Retorna cert si el número és positiu
}

int main(int argc, char *argv[]) {
    if (argc != 2) { // Comprova que es passa el paràmetre N
        fprintf(stderr, "Ús: ./replicant N > 0\n");
        exit(EXIT_FAILURE);
    }

    // Comprova que N sigui un enter positiu
    if (!valida_enter_positiu(argv[1])) {
        fprintf(stderr, "Error: N ha de ser un enter major que
0.\n");
        exit(EXIT_FAILURE);
    }

    // Assigna el valor de N
    int N = atoi(argv[1]);
    if (N > MAX_BUFFER) { // Comprova que N no superi el límit
màxim
        fprintf(stderr, "Error: N no pot excedir %d.\n",
MAX_BUFFER);
    }
}

```

```
exit(EXIT_FAILURE);  
}
```

Aquesta part del codi gestiona la lectura de dades des de l'entrada estàndard, el processament d'aquestes dades i la comunicació amb el servidor de manera que es calculi el resultat global de les operacions realitzades pels replicants.

Primer, es crida a la funció `dummy_init(SERVNAME, MAX_BUFFER)` per inicialitzar la connexió amb el servidor utilitzant el nom del servidor (`SERVNAME`) i la mida màxima del buffer (`MAX_BUFFER`). Això prepara el programa per començar a interactuar amb el servidor de dades per realitzar les operacions que s'indiquen.

A continuació, es crea un buffer anomenat `buffer` de mida `N`, que és el valor passat com a paràmetre al programa. Aquest buffer s'utilitzarà per emmagatzemar temporalment les dades llegides de l'entrada estàndard. També es defineixen les variables `bytes_llegits`, que serà utilitzada per emmagatzemar el nombre de bytes llegits, i `resultat`, inicialitzada a 0, que acumularà els resultats de les operacions realitzades.

El programa entra en un bucle `while` on llegeix les dades de l'entrada estàndard mitjançant la funció `read(STDIN_FILENO, buffer, N)`. Aquesta funció llegeix fins a `N` bytes de l'entrada estàndard i els desa a `buffer`. El bucle es repeteix mentre hi hagi dades per llegir (és a dir, mentre `bytes_llegits` sigui superior a 0). Durant cada iteració del bucle, es crida la funció `dummy_calc(buffer, bytes_llegits)` per processar les dades llegides. El resultat retornat per aquesta funció es suma a la variable `resultat`, que acumula els resultats de totes les operacions realitzades fins al moment.

Si en algun moment la funció `read` retorna un valor negatiu, significa que ha hagut un error en la lectura de l'entrada estàndard. En aquest cas, es crida a la funció `gestiona_error` per gestionar l'error de manera robusta, mostrant un missatge descriptiu i finalitzant l'execució del programa.

Un cop totes les dades han estat processades i el resultat ha estat acumulat, es crida a la funció `dummy_end()`, que finalitza la connexió amb el servidor de dades. Això allibera els recursos associats a la connexió i tanca qualsevol interacció pendent amb el servidor.

Finalment, el programa retorna el valor de resultat al procés principal mitjançant la funció `exit(resultat)`. Aquest valor serà utilitzat com a codi de sortida del programa, indicant l'estat de l'execució i el resultat de les operacions realitzades pels replicants.

En resum, aquesta part del codi s'encarrega de llegir les dades de l'entrada estàndard, processar-les mitjançant el servidor, acumular els resultats i finalitzar la connexió amb el servidor, retornant el resultat final al procés principal.

```
dummy_init(SERVNAME, MAX_BUFFER); // Inicialitza la connexió al
servidor

char buffer[N]; // Crea un buffer de mida N
ssize_t bytes_llegits;
int resultat = 0;

// Llegeix i processa les dades de l'entrada estàndard
while ((bytes_llegits = read(STDIN_FILENO, buffer, N)) > 0) {
    resultat += dummy_calc(buffer, bytes_llegits); // Processa
les dades llegides
}

if (bytes_llegits < 0) { // Comprova si hi ha hagut error en
la lectura
    gestiona_error("Error llegint de l'entrada estàndard");
}

dummy_end(); // Finalitza la connexió al servidor

exit(resultat); // Retorna el resultat al procés principal
}
```

CODI CAPITOL.C:

Aquesta codi s'encarrega de gestionar la recepció i manipulació de la senyal SIGUSR1, la lectura de dades des de l'entrada estàndard, la seva escriptura a la sortida estàndard i la gestió d'errors.

Primer, es declara la variable `senyal_rebuda` com a `volatile sig_atomic_t` per indicar si s'ha rebut una senyal. Aquesta variable és modificada per la funció gestora quan es rep la senyal SIGUSR1. La paraula clau `volatile` assegura que el compilador no optimitzi aquesta variable, ja que es pot actualitzar de manera asincrònica a partir d'una senyal.

A continuació, es defineix la funció `gestiona_senyal`, que actua com a gestor de la senyal SIGUSR1. Quan el programa rep aquesta senyal, el gestor actualitza la variable `senyal_rebuda` a 1, indicant que la senyal ha estat rebuda i processada. La funció `signal(SIGUSR1, gestiona_senyal)` s'utilitza per assignar aquest gestor a la senyal SIGUSR1.

Després, es defineix la funció `gestiona_error` per gestionar qualsevol error que pugui ocórrer durant l'execució del programa. Aquesta funció utilitza `perror` per mostrar un missatge d'error a la sortida d'errors i després finalitza l'execució del programa amb el codi `exit(EXIT_FAILURE)`.

Al cos de la funció `main`, es configura el gestor de la senyal SIGUSR1 mitjançant `signal(SIGUSR1, gestiona_senyal)`. Si hi ha un error en la configuració del gestor, es crida a la funció `gestiona_error`.

Tot seguit, el programa envia una senyal SIGUSR1 al procés pare utilitzant la funció `kill(getppid(), SIGUSR1)`. Si aquesta operació falla, també es crida a `gestiona_error` per gestionar l'error.

A continuació, el programa entra en un bucle on llegeix les dades de l'entrada estàndard mitjançant la funció `read(STDIN_FILENO, buffer, sizeof(buffer))` i les escriu a la sortida estàndard amb la funció `write(STDOUT_FILENO, buffer, bytes_llegits)`. Aquest bucle es repeteix fins que no hi ha més dades per llegir. Si es produeix un error en llegir o escriure les dades, es crida a la funció `gestiona_error` per gestionar l'error de manera robusta.

Si la funció `read` retorna un valor negatiu, el programa detecta que hi ha hagut un error en llegir des de l'entrada estàndard i es crida a `gestiona_error` per gestionar aquest error.

Finalment, el programa finalitza retornant 0, indicant que l'execució ha estat correcta. En resum, aquesta part del codi es dedica a gestionar la senyal `SIGUSR1`, llegir dades de l'entrada estàndard, escriure-les a la sortida estàndard i gestionar els errors que puguin sorgir durant aquests processos.

```
/* Programa robust per capitol.c */
#include "dummy4.h"
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>
#include <fcntl.h>
#include <errno.h>

volatile sig_atomic_t senyal_rebuda = 0; // Variable per indicar
si s'ha rebut la senyal

// Gestor de la senyal SIGUSR1
void gestiona_senyal(int senyal) {
    if (senyal == SIGUSR1) {
        senyal_rebuda = 1; // Marca que la senyal ha estat rebuda
    }
}

// Funció per gestionar errors
void gestiona_error(const char *missatge) {
    perror(missatge); // Mostra l'error
    exit(EXIT_FAILURE); // Termina el programa amb codi d'error
}

int main() {
    // Assigna el gestor de la senyal
    if (signal(SIGUSR1, gestiona_senyal) == SIG_ERR) {
        gestiona_error("Error assignant el gestor de senyals");
    }
}
```

```

// Envia una senyal SIGUSR1 al procés pare
if (kill(getppid(), SIGUSR1) < 0) {
    gestiona_error("Error enviant senyal al procés pare");
}

char buffer[1024];
ssize_t bytes_llegits;

// Llegeix dades de l'entrada estàndard i les escriu a la
sortida estàndard
while ((bytes_llegits = read(STDIN_FILENO, buffer,
sizeof(buffer))) > 0) {
    if (write(STDOUT_FILENO, buffer, bytes_llegits) !=
bytes_llegits) { // Comprova que s'escriu tot
        gestiona_error("Error escrivint a la sortida
estàndard");
    }
}

if (bytes_llegits < 0) { // Comprova si hi ha hagut error en
la lectura
    gestiona_error("Error llegint de l'entrada estàndard");
}

return 0;
}

```

COMPROVACIÓ DEL CODI:

```
printf("MISSION: %s\n", dummy);
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o tt tt.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o capitol capitol.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o replicant replicant.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ ./tt 2 3

*** CLIENT: Connecting server at [147.83.159.200]:[9900]

*** CHECKING: ENIGMA3 and SILVERKEY ...
[OK]: correct ENIGMA3 and SILVERKEY!!!

*** CLIENT: identification

*** CLIENT: sending teamname ...

[CAPITOL]: CLIENT davidperezcruz ALREADY REGISTERED

Waiting to read TimeBank....

*** CLIENT: Connecting server at [147.83.159.200]:[9999] ready to receive data
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
MISSION: SUCCESSFUL!!.GOLDKEY:0xdc192
```